Quantization

Post-Training Quantization (PTQ) 중심

논문 읽기를 위한 초급 개념서

이 개념서는 Neural Network Quantization, 특히 Post-Training Quantization (PTQ) 논문을 읽기 위한 기초 수학·이론·알고리즘 개념을 다룹니다.

목차

1장 **Quantization이란 무엇인가?** — *기본 개념과 동기*
2장 수치 표현의 기초 — *부동소수점 vs 고정소수점, 비트 체계*
3장 **Quantization의 수학적 기초** — *스케일링, 영점, 클리핑*
4장 **Quantization 분류 체계** — *PTQ / QAT / 비트 범위 / 방향 분류*
5장 **Post-Training Quantization (PTQ) 심화** — *알고리즘, 기법, 파이프라인*
6장 핵심 오차 지표와 보정 기법 — *MSE, SQNR, Calibration*
7장 채널별·레이어별 양자화 전략 — *Per-tensor vs Per-channel*
8장 혼합 정밀도 양자화 — *Mixed-Precision Quantization*
9장 주요 **PTQ** 논문 키워드 가이드 — *논문 읽기를 위한 용어 정리*
10장 공부 로드맵 및 추천 자료 — *다음 단계로 나아가기*

1 장. Quantization 이란 무엇인가?

1.1 등장 배경

딥러닝 모델은 수백만~수천억 개의 파라미터(가중치, weight)로 이루어집니다. 학습 단계에서는 32비트 부동소수점(FP32)으로 파라미터를 저장하는 것이 일반적입니다. 그러나 실제 배포(inference) 환경에서는 다음과 같은 제약이 존재합니다.

- 메모리 한계: 스마트폰, 엣지 디바이스는 RAM 이 수 GB 에 불과합니다.
- 연산 속도: FP32 곱셈은 INT8 곱셈보다 수 배 느립니다.
- 전력 소비: 고정밀도 연산은 에너지 소모가 큽니다.

> *Quantization 은 이 문제를 해결하기 위해, 모델의 가중치와 활성화 값을 낮은 비트 수(예: 8 비트, 4 비트)로 표현하는 기술입니다. 정밀도를 줄이는 대신 속도와 메모리를 절약합니다.*

1.2 Quantization 의 핵심 아이디어

연속적인(또는 고정밀도) 실수 값을 이산적인(discrete) 정수 값으로 매핑하는 과정입니다. 예를 들어, [-1.5, 2.3, -0.7, 0.9] 같은 FP32 배열을 [-128, 127, -45, 58] 같은 INT8 배열로 변환합니다.

구분	FP32 (기존)	INT8 (양자화 후)
저장 크기	4 bytes / 파라미터	1 byte / 파라미터
메모리 절약	기준	약 4배 절약
연산 속도	기준	2~4배 가속
정확도 손실	없음 (기준)	소량 (잘 설계 시 <1%)
적용 시점	항상	학습 후 또는 학습 중

1.3 Quantization vs Pruning vs Knowledge Distillation

모델 경량화 기법에는 여러 종류가 있으며, Quantization은 그 중 하나입니다.

기법	방법	장점	단점
Quantization	가중치 비트 수 축소	구현 용이, HW 친화적	정밀도 손실 가능
Pruning	불필요한 가중치 제거	구조적 압축 가능	재학습 필요
Knowledge Distillation	작은 모델이 큰 모델 모	정확도 유지 우수	학습 비용 높음

기법	방법	장점	단점
	방		

2 장. 수치 표현의 기초

2.1 부동소수점 (Floating-Point)

현대 딥러닝에서 기본으로 사용하는 FP32(단정밀도)는 32비트로 하나의 실수를 표현합니다.

필드	비트 수	역할
Sign (부호)	1 bit	양수(0) / 음수(1)
Exponent (지수)	8 bit	값의 크기(스케일) 결정
Mantissa (가수)	23 bit	유효 숫자 결정

표현 값 = (-1)^sign × 2^(exponent-127) × (1.mantissa)

FP16 (반정밀도): 부호 1 + 지수 5 + 가수 10 = 16비트. 표현 범위 좁고 정밀도 낮음.

BF16 (Brain Float 16): 부호 1 + 지수 8 + 가수 7 = 16비트. FP32와 동일한 지수 범위를 가져 학습에 유리.

2.2 고정소수점 (Fixed-Point) & 정수 표현

Quantization에서는 주로 정수형(INT)을 사용합니다. 정수형은 소수점 위치가 고정되어 있어 연산이 빠릅니다.

타입	비트	범위	주요 용도
INT8	8 bit	-128 ~ 127 (signed)	가중치·활성화 양자화 표준
UINT8	8 bit	0 ~ 255 (unsigned)	ReLU 이후 활성화 값
INT4	4 bit	-8 ~ 7 (signed)	공격적 압축, LLM 분야
INT2 / Binary	1~2 bit	-1~1 / {0,1}	극단적 압축 연구

2.3 비트 수와 정보 손실의 관계

사용 가능한 이산 값의 개수는 2^n입니다. 비트가 줄어들수록 표현 가능한 값이 기하급수적으로 감소합니다.

- FP32: 2^32 ≈ 42 억 가지
- INT8: 2^8 = 256 가지
- INT4: 2^4 = 16 가지

- INT2: 2^2 = 4 가지

> *비트 수가 낮을수록 압축률은 높아지지만 정보 손실(양자화 오차)이 커집니다. PTQ 연구의 핵심은 이 trade-off를 최소화하는 것입니다.*

3 장. Quantization 의 수학적 기초

3.1 기본 Quantization 공식

Quantization의 핵심은 실수 값 r을 정수 q로 변환하는 함수(Q)와, 다시 근사 실수로 복원하는 역함수(Q^{-1})입니다.

> *$Q(r) = round(r / S) + Z$ (Quantize)*

> *$Q^{-1}(q) = S \times (q - Z)$ (Dequantize / 근사 복원)*

S (Scale): 실수 범위를 정수 범위로 매핑하는 스케일 팩터. 양수 실수값.
Z (Zero-point): 실수 0이 매핑되는 정수 값. Asymmetric에서 0이 아닐 수 있음.
round(): 반올림 연산 (가장 가까운 정수로).

3.2 Symmetric vs Asymmetric Quantization

영점(Z)의 위치에 따라 두 가지 방식으로 나뉩니다.

방식	Zero-point (Z)	수식	장점	단점
Symmetric	Z = 0	q = round(r/S)	연산 단순, 하드웨어 친화적	음수·양수 범위 비대칭 손실
Asymmetric	Z ≠ 0 (정수)	q = round(r/S) + Z	전체 값 범위 활용	Zero-point 저장 오버헤드

Symmetric 예시: [-2.0, 2.0] 범위를 INT8 [-128, 127]에 매핑 시

> *$S = 2.0 / 127 \approx 0.01575$, $Z = 0$ $r = 1.5 \rightarrow q = round(1.5 / 0.01575) \approx 95$*

Asymmetric 예시: [0, 2.0] 범위를 UINT8 [0, 255]에 매핑 시

> *$S = 2.0 / 255 \approx 0.00784$, $Z = 0$ $r = 1.5 \rightarrow q = round(1.5 / 0.00784) \approx 191$*

3.3 Clipping 과 Saturation

실수 분포의 최솟값~최댓값(min/max)을 그대로 사용하면 이상치(outlier)에 의해 분해능(granularity)이 낮아집니다. 이를 해결하기 위해 클리핑 범위 [α, β]를 설정합니다.

- α, β 범위를 벗어나는 값은 경계값으로 포화(saturation)됩니다.
- 클리핑 범위를 좁힐수록 분해능이 높아지지만 클리핑 오차가 증가합니다.
- 클리핑 범위를 넓힐수록 클리핑 오차는 줄지만 분해능이 낮아집니다.

> *MSE 최적 클리핑: argmin_{α,β} E[(r - Q(r))²] — 이것이 많은 PTQ 논문의 핵심 최적화 목표입니다.*

3.4 양자화 오차 (Quantization Error)

양자화 오차는 원본 값과 복원 값의 차이입니다.

> $\varepsilon = r - Q^{-1}(Q(r)) = r - S \times (round(r/S) - Z)$

오차는 두 종류로 분류됩니다.

반올림 오차 (Rounding Error): round() 연산에서 발생하는 ±0.5S 범위의 오차.

클리핑 오차 (Clipping Error): 범위를 벗어난 값이 포화될 때 발생하는 오차.

4 장. Quantization 분류 체계

4.1 적용 시점 기준: PTQ vs QAT

구분	Post-Training Quantization (PTQ)	Quantization-Aware Training (QAT)
적용 시점	학습 완료 후	학습 중 (Forward pass에 적용)
데이터 필요량	소량 (캘리브레이션용)	전체 학습 데이터
구현 난이도	낮음	높음
정확도 유지	중간~높음 (비트 수에 따라)	매우 높음
주요 적용처	빠른 배포, 대형 모델	고정밀도 요구 응용

> *PTQ는 이미 학습된 모델을 재학습 없이 빠르게 압축할 수 있어, 대형 언어 모델(LLM) 양자화의 주류 방법입니다.*

4.2 양자화 대상 기준

Weight-only Quantization: 가중치(weight)만 양자화. 활성화는 FP로 유지. 메모리 절약 우선.

Weight + Activation Quantization (W/A Quantization): 가중치와 활성화 모두 양자화. 연산 가속 극대화.

KV Cache Quantization: LLM의 Key-Value 캐시를 양자화하여 메모리 절약.

4.3 양자화 세분성 (Granularity)

레벨	설명	S, Z 개수	특징
Per-tensor	텐서 전체에 단일 S, Z 사용	1개씩	최단순, 정확도 손실 가능
Per-channel	채널(출력 뉴런)마다 별도 S, Z	채널 수만큼	가중치에 주로 사용, 정확도 향상
Per-token	LLM의 토큰마다 별도 S, Z	토큰 수만큼	활성화 이상치 대응
Per-group	그룹(예: 128개)마다 별도 S, Z	파라미터수/그룹크기	INT4 등 초저비트에서 사용

4.4 비트 수 기준 분류

- High-bit (INT8, FP8): 정확도 손실 거의 없음. 현재 산업 표준.
- Low-bit (INT4): 정확도 손실 주의. LLM 분야에서 활발히 연구.
- Ultra-low bit (INT2, Binary/Ternary): 극단적 압축. 특수 연구 영역.

5 장. Post-Training Quantization (PTQ) 심화

5.1 PTQ 파이프라인 개요

PTQ의 일반적인 처리 흐름은 다음과 같습니다.

1. 사전학습 모델(FP32) 로드
2. 소량의 캘리브레이션 데이터(Calibration Data)를 Forward pass
3. 각 레이어의 가중치·활성화 분포(min, max, histogram) 수집
4. 최적 Scale / Zero-point 계산 (Calibration)
5. 가중치를 저비트 정수로 변환 및 저장
6. (선택) 레이어별 오차 보정 (GPTQ, AWQ 등)
7. 평가 및 배포

5.2 Calibration 방법론

캘리브레이션은 각 텐서의 클리핑 범위를 결정하는 과정입니다.

5.2.1 Min-Max Calibration

가장 단순한 방법. 캘리브레이션 데이터의 실제 최솟값·최댓값을 클리핑 범위로 사용합니다.

- 장점: 구현 단순.
- 단점: 이상치(outlier)에 매우 민감. 단 하나의 극단값이 전체 분해능을 낮춤.

5.2.2 Percentile Calibration

분포의 99.9% 또는 99.99% 백분위수를 클리핑 범위로 사용합니다. 이상치의 영향을 줄입니다.

5.2.3 Entropy (KL Divergence) Calibration

원본 FP32 분포와 양자화 후 분포 간의 KL Divergence를 최소화하는 클리핑 범위를 탐색합니다. TensorRT에서 사용하는 방식입니다.

> *$KL(P \| Q) = \Sigma P(x) \cdot \log(P(x) / Q(x))$ — 이 값을 최소화하는 클리핑 범위를 grid search로 찾음*

5.2.4 MSE Calibration

양자화 오차의 MSE를 직접 최소화하는 클리핑 범위를 탐색합니다.

> *argmin_{α} E[(r - Q(clip(r, -α, α)))²]*

5.3 가중치 등가 변환 (Weight Equalization)

DFQ(Data-Free Quantization) 논문(Nagel et al., 2019)에서 제안된 기법입니다. 서로 다른 채널 간의 가중치 분포 스케일 불균형을 줄여 양자화 친화적으로 만듭니다.

> *핵심 아이디어: 두 연속 레이어에서 W1의 채널 스케일과 W2의 채널 스케일을 교환해도 네트워크 출력은 동일. 이를 이용해 모든 채널의 가중치 범위를 균등화.*

5.4 Bias Correction

가중치를 양자화하면 출력 텐서의 기댓값(mean)이 달라질 수 있습니다. Bias correction은 이 편향(bias shift)을 보정하는 기법입니다.

> *Δbias = E[W·x] - E[Q(W)·x] → bias += Δbias*

5.5 GPTQ (Accurate Post-Training Quantization)

Frantar et al.(2022)이 제안한 LLM 전용 PTQ 기법으로, 현재 LLM 양자화의 사실상 표준입니다.

핵심 아이디어 - OBS (Optimal Brain Surgeon): 가중치를 하나씩 양자화하면서 발생하는 오차를, 나머지 가중치를 2차 헤시안(Hessian) 정보를 활용해 보정합니다.

- Hessian 행렬 $H = 2X \cdot X^T$ (X: 입력 활성화)
- 하나의 가중치를 양자화 후, 나머지 가중치 업데이트: $\delta W = -(w_q - w) / [H^{-1}]_{qq} \cdot H^{-1}_q$
- 행렬 역산은 Cholesky 분해로 효율화.
- GPU 에서 수십억 파라미터 모델을 수 시간 내에 양자화 가능.

> *GPTQ를 사용하면 Llama-2-70B 규모 모델도 INT4로 양자화 시 성능 손실 < 1% 수준이 가능합니다.*

5.6 AWQ (Activation-aware Weight Quantization)

Lin et al.(2023)이 제안. 활성화 이상치(outlier)에 대응하기 위해 가중치를 채널별로 스케일링한 후 양자화합니다.

핵심 관찰: LLM에서 입력 활성화의 1% 미만 채널이 매우 큰 이상치 값을 가지며, 이 채널이 성능에 지배적 영향을 미칩니다.

해결책: 중요 채널의 가중치를 미리 스케일업 → 양자화 → 스케일다운으로 이상치 영향을 완화합니다.

5.7 SmoothQuant

Xiao et al.(2022)이 제안. W/A 동시 양자화를 위해 활성화의 이상치를 가중치로 '이전(migration)'합니다.

> *핵심 공식: $Y = (X / s) \cdot (W \cdot diag(s))$ — s는 채널별 스케일 팩터*

활성화를 나누고 가중치를 곱해서 활성화의 이상치를 제거. 결과적으로 양자화가 쉬운 형태로 변환.

6 장. 핵심 오차 지표와 보정 기법

6.1 양자화 품질 평가 지표

지표	수식	해석
MSE	$E[(r - \hat{r})^2]$	오차의 평균 제곱. 낮을수록 좋음.
SQNR	$10 \cdot \log_{10}(E[r^2] / E[\varepsilon^2])$ (dB)	신호 대 잡음비. 높을수록 좋음.
PSNR	$20 \cdot \log_{10}(\max / \mathrm{RMSE})$	이미지 분야에서 많이 사용.
PPL (Perplexity)	언어 모델 전용 지표	낮을수록 언어 모델 품질 높음.
Top-1 Accuracy	분류 정확도	FP32 대비 하락 폭이 작을수록 우수.

6.2 레이어별 오차 전파

양자화 오차는 한 레이어에서 끝나지 않고 다음 레이어로 전파됩니다. 특히 잔차 연결(residual connection)이 있는 Transformer 구조에서는 누적 오차가 더 크게 나타날 수 있습니다.

> *실용적 접근: 각 레이어를 독립적으로 양자화할 때는 해당 레이어 출력의 MSE를 최소화하는 방향으로 최적화. (Layer-wise error minimization — GPTQ, OPTQ의 기본 가정)*

6.3 Activation Outlier 문제

LLM (특히 OPT, LLaMA 등)의 활성화 값에서는 특정 채널에 매우 큰 이상치가 존재합니다. 이러한 이상치는 다음과 같은 문제를 야기합니다.

- Per-tensor 양자화 시 대부분의 정상 값들이 좁은 정수 범위에 몰려 분해능 손실.
- INT8 이하 비트에서 심각한 정확도 하락.

대응 기법: AWQ, SmoothQuant, LLM.int8() (bitsandbytes) 등이 이 문제를 해결합니다.

7 장. 채널별·레이어별 양자화 전략

7.1 Per-tensor vs Per-channel 상세 비교

Per-channel quantization은 각 출력 채널(필터)마다 독립적인 S와 Z를 사용합니다. 가중치의 채널 간 스케일 차이가 크게 다른 경우 유리합니다.

> *예: Conv 레이어의 가중치 채널 0~63 각각에 독립적 Scale을 적용하면, 채널별로 최적화된 분해능을 사용할 수 있습니다.*

PyTorch의 per_channel_affine quantization 모드가 이에 해당합니다.

7.2 Group Quantization

Per-channel보다 더 세밀한 방법으로, 가중치를 특정 그룹 크기(예: 128)로 나누어 각 그룹에 독립적인 S, Z를 적용합니다.

- GPTQ, AWQ, GGUF 등 4 비트 LLM 양자화에서 널리 사용.
- 그룹 크기 128: 성능/오버헤드 trade-off 의 현실적 표준.
- 그룹 크기가 작을수록 성능↑, 메타데이터 오버헤드↑.

7.3 레이어 감도 분석 (Layer Sensitivity Analysis)

모든 레이어가 양자화에 동일하게 민감하지 않습니다. 일부 레이어는 저비트 양자화에 매우 민감하고, 다른 레이어는 강건합니다.

첫 번째와 마지막 레이어: 입력/출력에 가까울수록 오차 영향이 크므로 FP16 유지가 권장됩니다.

Attention QKV 프로젝션: LLM에서 이상치가 많아 양자화 어려움.

8 장. 혼합 정밀도 양자화 (Mixed-Precision Quantization)

8.1 개념

모든 레이어를 같은 비트 수로 양자화하지 않고, 레이어의 감도에 따라 서로 다른 비트 수를 할당하는 방법입니다.

> *민감한 레이어 → FP16 또는 INT8 유지 덜 민감한 레이어 → INT4 또는 INT2 적용 → 평균 비트 수 감소 + 정확도 유지*

8.2 비트 배분 전략

수동 설정: 경험적 규칙 또는 민감도 분석으로 레이어별 비트 수 결정.

자동화 (AutoQ, HAQ 등): 강화학습(RL)이나 NAS(신경망 구조 탐색)를 사용해 최적 비트 배분 자동 탐색.

GPTQ + 일부 레이어 FP16: 실용적인 절충안으로 많이 사용됨.

8.3 현실적 적용 예시 (LLM)

레이어 유형	일반적 비트 수 배정	이유
Embedding layer	FP16 또는 INT8	어휘 표현 손실 민감
Attention QKV	INT8 권장	이상치 多, 민감
FFN (MLP)	INT4 가능	파라미터 수 많고 덜 민감
LM Head (출력층)	FP16	확률 분포 민감

9 장. 주요 PTQ 논문 키워드 가이드

9.1 반드시 알아야 할 핵심 용어

용어	의미
Quantization Grid	양자화 가능한 이산 값들의 집합
Rounding Error	round() 연산에서 발생하는 오차 (최대 ±0.5·S)
Clipping Error	범위 밖 값이 경계로 포화될 때 발생하는 오차
Calibration	스케일/영점 결정을 위한 데이터 통계 수집 과정
Dequantization	정수 → 근사 실수로 복원하는 과정
Fake Quantization	QAT에서 Forward에는 Q-DQ 적용, 실제 저장은 FP32로 유지
W4A16	가중치 INT4, 활성화 FP16을 의미하는 표기법
W8A8	가중치 INT8, 활성화 INT8
PTQ4ViT	Vision Transformer 전용 PTQ 기법
GGUF / GGML	llama.cpp에서 사용하는 LLM 양자화 포맷
bitsandbytes	LLM.int8() 구현 라이브러리. 이상치를 FP16으로 분리 처리
Hessian	2차 편미분 행렬. GPTQ에서 오차 보정에 사용
OBC (Optimal Brain Compression)	Hessian 기반 순차 양자화 기법의 이론적 기반

9.2 자주 등장하는 약어

약어	전체 명칭	설명
PTQ	Post-Training Quantization	학습 후 양자화
QAT	Quantization-Aware Training	학습 중 양자화
LSQ	Learned Step Size Quantization	스케일 팩터를 학습으로 최적화
ADAROUND	Adaptive Rounding	반올림 방향을 학습으로 결정
BRECQ	Block-wise Reconstruction Quantization	블록 단위 재구성 최적화 PTQ
OBQ	Optimal Brain Quantization	GPTQ의 전신, 가중치별 순차 보정
ZeroQuant	Zero-overhead Quantization	커널 수준 INT8 최적화

약어	전체 명칭	설명
SpQR	Sparse-Quantized Representation	이상치 가중치를 별도 FP로 저장

10 장. 공부 로드맵 및 추천 자료

10.1 단계별 학습 로드맵

단계	내용	목표
1단계 (기초)	이 개념서 + PyTorch Quantization 공식 튜토리얼	기본 API 사용, PTQ 파이프라인 이해
2단계 (핵심 논문)	DFQ(2019), AdaRound(2020), BRECQ(2021)	알고리즘 구현 원리 이해
3단계 (LLM 양자화)	GPTQ(2022), SmoothQuant(2022), AWQ(2023)	LLM 특수성 이해
4단계 (최신 연구)	QuaRot, QServe, AQLM 등 최신 논문	연구 트렌드 파악

10.2 필독 논문 목록

기초 / 이론

- Nagel et al. (2019) — Data-Free Quantization Through Weight Equalization and Bias Correction
- Nagel et al. (2020) — Up or Down? Adaptive Rounding for Post-Training Quantization (AdaRound)
- Li et al. (2021) — BRECQ: Pushing the Limit of Post-Training Quantization by Block Reconstruction

LLM 양자화

- Frantar et al. (2022) — GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers
- Xiao et al. (2022) — SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models
- Lin et al. (2023) — AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration
- Dettmers et al. (2022) — LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale

서베이 논문 (전체 조망)

- Gholami et al. (2021) — A Survey of Quantization Methods for Efficient Neural Network Inference

10.3 추천 실습 도구

도구	설명	링크
PyTorch Quantization	공식 PTQ/QAT API	pytorch.org/docs/stable/quantization
bitsandbytes	LLM INT8/NF4 양자화	github.com/TimDettmers/bitsandbytes
AutoGPTQ	GPTQ 구현 라이브러리	github.com/PanQiWei/AutoGPTQ
llama.cpp	GGUF 포맷 LLM 양자화 및 추론	github.com/ggerganov/llama.cpp
Hugging Face Optimum	다양한 양자화 통합 API	huggingface.co/docs/optimum

10.4 논문을 읽기 전 체크리스트

8. 사용된 양자화 방식: PTQ 인가, QAT 인가?
9. 양자화 대상: Weight 만인가, Weight+Activation 인가?
10. 비트 수 및 그래뉼래리티: W4A16 인가, per-channel 인가, group 인가?
11. 캘리브레이션 방법: Min-Max, MSE, KL, 아니면 별도 기법?
12. 베이스라인 모델: CNN 인가, Transformer 인가, LLM 인가?
13. 평가 지표: Accuracy, Perplexity, SQNR 중 무엇?
14. 핵심 기여: 새로운 클리핑 기법? 가중치 보정? 이상치 처리?

> *이 개념서에서 다룬 내용을 이해했다면, 주요 PTQ 논문의 Abstract와 Introduction을 막힘 없이 읽을 수 있습니다. 실제 코드 구현과 논문 읽기를 병행하면 이해가 빠릅니다.*

— 끝 —